DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III SEMESTER B. TECH (2024-2025)

SUBJECT: Data Structures (CSE 2122)

MID-TERM EXAMINATION

Date of exam: 24-09-2024 **Time:** 10:45 AM to 12:15 PM

Q. No.	Questions	M	CO (CLO)	LO	BL

1.	What will be the output of the following C code?
	#include<stdio.h>
	enum exam
	{
	a=2,b=3.56
	};
	enum exam s;
	void main()
	{
	printf("%d%d",a,b);
	}
	
	1. 2 3
	2. run-time error
	3. 2 3.56
	4. comple-time error
		0.5	1	1	2

Ans	**4. comple-time error**				
2.	How do you declare an array of pointers to integers? 1. int* arr[]; 2. arr int*[]; 3. int arr*[]; 4. int[]* arr;	0.5	1	1	2
Ans	**1. int* arr[];**				
3	What is the postfix expression for the corresponding infix expression? a+b*c+(d*e)^k 1. abc+*de*+^k 2. abc*+de*k^+ 3. abc+*de*+k+^ 4. abc*+(de)*+k^	0.5	2	1	3

Ans	2. abc*+de*k^+				
4	A normal queue, if implemented using an array of size MAX_SIZE, gets full when? 1. Rear = MAX_SIZE – 1 2. Front = (rear + 1)mod MAX_SIZE 3. Front = rear + 1 4. Rear = front	0.5	2	1	2
Ans	1. Rear = MAX_SIZE – 1				

5	What does the following function do for a given Linked List with the first node as the head?
	`void fun1(struct node* head)`
	`{`
	`  if(head == NULL)`
	`  return;`
	`  fun1(head->next);`
	`  printf("%d  ", head->data);`
	`}`
	1. Prints all nodes of linked list
	2. Prints all nodes of linked list in reverse order
	3. Prints alternate nodes of Linked List
	4. Prints alternate nodes in reverse order
		0.5	3	1	3

Ans	2. Prints all nodes of linked list in reverse order				
6	How many pointers must be updated when inserting a node at the end of a singly linked list? 1. 1 2. 2 3. 3 4. None	0.5	3	1	3
Ans	1. 1				
7	Which of the following is true about linked list implementation of queue? 1. In enqueue operation, if new nodes are inserted at the beginning of linked list, then in dequeue operation, nodes must be removed from end.	0.5	3	1	3

	2. In enqueue operation, if new nodes are inserted at the end, then in dequeue operation, nodes must be removed from the beginning. 3. Both A and B 4. None of the above				
Ans	3. Both A and B				

8	The following steps in linked stack results in which kind of operation?. Here the function getnode() is to obtain a new node for the list.
	
	newnode = getnode();
	newnode->data = item;
	newnode->link = list;
	list= newnode;
	
	1. push operation at the last
	2. push operation at the front
	3. push operation at the middle
	4. Both A and B
		0.5	3	1	2

Ans	**2. push operation at the front**				

9	Analyze the code snippet given below and identify what it does. Here the function new_Node() is to obtain a new node for the list.
	
	Node* head = new_Node();
	Node* tail = new_Node();
	head->next = tail;
	tail->next = head;
	Node* current = head;
	do { current = current->next;
	} while (current != head);
	
	1. It creates a doubly linked list.
	2. It creates a circular linked list and traverses it once.
	3. It creates a linear linked list and traverses it twice.
	4. It creates a circular linked list and traverses it indefinitely.
		0.5	3	1	3

Ans	2. It creates a circular linked list and traverses it once.				

10	Analyze the following C code snippets and identify the correct one to fully erase the dynamically allocated memory to a polynomial represented using a linked list, where ***poly*** is a pointer to the first node of the list.
	
	1. free(poly);
	2. while (poly != NULL)
	{
	struct Node* temp = poly;
	poly = poly->next;
	free(temp);
	}
	3. delete(poly)
	4. while (poly != NULL)
	{
	struct Node* temp = poly;
	poly = poly->next;
	delete(temp);
	}
		0.5	3	1	4

Ans	2. while (poly != NULL)
	{
	struct Node* temp = poly;
	poly = poly->next;
	free(temp);
	}				

11	Design the C functions to perform the following operations on a polynomial, where the polynomial is of the form: $P(x)=a_n.x^n + a_{n-1}.x^{n-1} + \ldots.. + a_1.x + a_0$ 1. **Insert** a new term (coefficient and exponent) into the polynomial 2. **Add** two polynomials and return the resulting polynomial	4	3	1	6
Ans	**Scheme:** Node structure declaration - **0.5 M** **Insert a new term** (coefficient and exponent) into the polynomial. **-1 M** **Add two polynomials** and return the resulting polynomial. **-2.5 M** **Solution:** #include <stdio.h> #include <stdlib.h> #include<math.h> // Define a node for the linked list				

```
struct Term {
    int coefficient;
    int exponent;
    struct Term* next;
};

// Define the polynomial structure
struct Polynomial {
    struct Term* head;
};
// Function to create a new term
struct Term* create_term(int coefficient, int exponent) {
    struct Term* new_term = (struct Term*)malloc(sizeof(struct Term));
    new_term->coefficient = coefficient;
    new_term->exponent = exponent;
    new_term->next = NULL;
    return new_term;
}

// Function to insert a term into the polynomial
void insert_term(struct Polynomial* poly, int coefficient, int exponent) {
    struct Term* new_term = create_term(coefficient, exponent);

    if (poly->head == NULL || poly->head->exponent < exponent) {
        new_term->next = poly->head;
        poly->head = new_term;
        return;
    }
    struct Term* current = poly->head;
    while (current->next != NULL && current->next->exponent > exponent) {
        current = current->next;
    }
    if (current->exponent == exponent) {
        current->coefficient += coefficient;
    } else {
        new_term->next = current->next;
        current->next = new_term;
    }   }

// Function to evaluate the polynomial for a given value of x
double evaluate(struct Polynomial* poly, double x) {
    double result = 0.0;
    struct Term* current = poly->head;
    while (current != NULL) {
```

```
        result += current->coefficient * pow(x, current->exponent);
        current = current->next;
    }
    return result;
}

// Function to add two polynomials
struct Polynomial* add_polynomials(struct Polynomial* poly1, struct Polynomial*
poly2) {
    struct Polynomial* result = (struct Polynomial*)malloc(sizeof(struct Polynomial));
    result->head = NULL;

    struct Term* p1 = poly1->head;
    struct Term* p2 = poly2->head;

    while (p1 != NULL || p2 != NULL) {
        if (p1 == NULL) {
            insert_term(result, p2->coefficient, p2->exponent);
            p2 = p2->next;
        } else if (p2 == NULL) {
            insert_term(result, p1->coefficient, p1->exponent);
            p1 = p1->next;
        } else if (p1->exponent > p2->exponent) {
            insert_term(result, p1->coefficient, p1->exponent);
            p1 = p1->next;
        } else if (p1->exponent < p2->exponent) {
            insert_term(result, p2->coefficient, p2->exponent);
            p2 = p2->next;
        } else {
            insert_term(result, p1->coefficient + p2->coefficient, p1->exponent);
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    return result;
}
// Function to display the polynomial
void display(struct Polynomial* poly) {
    struct Term* current = poly->head;
    while (current != NULL) {
        printf("%dx^%d", current->coefficient, current->exponent);
        if (current->next != NULL) {
            printf(" + ");
        }
```

```
        current = current->next;
    }
    printf("\n");
}

int main() {
     int i,m,n,x,c,e;
    struct Polynomial poly1 = {NULL};
    struct Polynomial poly2 = {NULL};
printf("enter the number of terms in polynomial 1\n");
scanf("%d",&m);
for(i=0;i<m;i++)
  {
       printf("enter coefficient and exponent\n");
     scanf("%d%d",&c,&e);

    insert_term(&poly1, c, e);
  }
printf("enter the number of terms in polynomial 2\n");
scanf("%d",&n);
for(i=0;i<n;i++)
{
     printf("enter coefficient and exponent\n");
     scanf("%d%d",&c,&e);    // Polynomial 2: 3x^3 + 2x + 1
    insert_term(&poly2, c, e);
}
    // Display both polynomials
    printf("Polynomial 1: ");
    display(&poly1);

    printf("Polynomial 2: ");
    display(&poly2);
    printf("enter the value of x\n");
    scanf("%d",&x);
    printf("Evaluation of Polynomial 1 at x = %d: %.2f\n", x,evaluate(&poly1, x));

    // Add both polynomials and display the result
    struct Polynomial* sum = add_polynomials(&poly1, &poly2);
    printf("Sum of Polynomial 1 and Polynomial 2: ");
    display(sum);

    return 0;
}
```

12	Consider the following structure definition in C.
	struct Student
	{
	char name[50];
	int rollNumber;
	float marks;
	};
	Write a C function **createAndPrintStudents** that dynamically allocates memory for an array of Student structures, initializes the array with student data, and prints the details of each student in the format given below.
	
	Name: name, Roll Number: rollNumber, Marks: marks.
		3	1	1	4

Ans

Scheme:

Dynamic memory allocation - **1M**
Storing the data in the dynamically created structure object - **1M**
Printing all the students' information - **1M**

Solution:

```
//Sample solution
void createAndPrintStudents(int count) {
    struct Student* students = (struct Student*)malloc(count * sizeof(struct Student));
    if (students != NULL) {
        // Initializing student data
        for (int i = 0; i < count; i++) {
            printf("Enter name for student %d: ", i + 1);
            scanf("%s", students[i].name);
            printf("Enter roll number for student %d: ", i + 1);
            scanf("%d", &students[i].rollNumber);
            printf("Enter marks for student %d: ", i + 1);
            scanf("%f", &students[i].marks);
        }
        // Printing student data
        for (int i = 0; i < count; i++) {
            printf("Name: %s, Roll Number: %d, Marks: %.2f\n", students[i].name,
students[i].rollNumber, students[i].marks);
        }
        // Freeing allocated memory
        free(students);
    } else {
        printf("Memory allocation failed\n");
    }
}
```

```
int main() {                    //optional
    int count;
    printf("Enter the number of students: ");
    scanf("%d", &count);
    createAndPrintStudents(count);
    return 0;
}
```

13	Justify the need to convert infix expression to either prefix or postfix? Convert the infix expression A+(B-C*D)/E^F^G to its equivalent postfix by filling the below given table. (The ^ represents the power operator). Symbol scanned | Operation performed | Stack content | Postfix array | | | | | | | | | | | |	3	2	1	4

Ans

Scheme:

Justification for conversion – 1 M

Expression conversion with table – 2 M

Solution:

Converting infix expressions into **postfix/prefix** makes expression free from parentheses, precedence and associativity. 1m

Symbol scanned	Operation performed	Stack content	Postfix array
A	-	-	A
+	Push(+)	+	A
(	Push(( )	+ (	A
B	-	+ (	AB
-	Push(-)	+ (-	AB
C	-	+ (-	ABC
*	Push(*)	+ (- *	ABC
)	Pop and append to postfix array, Pop and append to postfix array, pop	+	ABC*-
D	--	+	ABC*-D
/	Push(/)	+/	ABC*-D
^	Push(^)	+/^	ABC*-D
^	Push(^)	+/^^	ABC*-D
--	Pop everything from stack and append it top postfix array		ABC*-D^^/+

2m

14	Write a C program to simulate a Car Parking System (CPS) with a maximum capacity of N parking slots, where no more than N-1 cars can be parked at a time. Each car is identified by its registration number and the owner's Aadhar ID. Vehicles enter the parking lot in order of their arrival and leave in the same order. If the parking lot is full when a new vehicle arrives, the system should notify that the parking lot is full. When all vehicles have exited, the system should indicate that the parking lot is empty. Cars can enter the parking lot if space becomes available due to the departure of other vehicles.	3	2	1	4

Ans

Scheme:

Structure definition – 1M
Circular queue operations – 1M
Main – 1M

Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include<string.h>
#define SIZE 3

// Structure definition for Queue
typedef struct car {
int adhar_no;
char reg_no[50];
}C;

typedef struct que
{
C c[SIZE];
int front;
int rear;
}Q;

// Function to check if the queue is full
int isFull(Q *q) {
if ((q->rear+1)%SIZE == q->front) return 1;
return 0;
}
// Function to check if the queue is empty
int isEmpty( Q *q) {
if (q->front == q->rear) return 1;
return 0;
}
// Function to add an element to the queue (enqueue)
void park( Q *q, int adhar, char reg_no[]) {
```

```
if (isFull(q)) {
printf("Car parking lot is full! Cannot park cars\n");
} else {
q->rear=(q->rear+1)%SIZE;
q->c[q->rear].adhar_no = adhar;
strcpy(q->c[q->rear].reg_no,reg_no);
}
}
// Function to remove an element from the queue (dequeue)
void remove_car(Q *q) {
int item;
if (isEmpty(q)) {
printf("Car parking lot is empty! Cannot remove cars\n");
return ;
} else {
q->front= (q->front+1)%SIZE;
printf("The adhar number is %d\t car registration number is %s\n",q->c[q-
>front].adhar_no,q->c[q->front].reg_no);}
return ;
}

// Function to display the elements of the queue
void display(Q q) {
if (isEmpty(&q)) {
printf("Parking lot is free \n");
} else {
int i;
printf("carks parked in the parking lot are \n");
for (i = (q.front+1); i !=q.rear; i=(i+1)%SIZE) {
printf("%d\t%s\n", q.c[i].adhar_no,q.c[i].reg_no);
}
printf("%d\t%s", q.c[i].adhar_no,q.c[i].reg_no);
printf("\n");
}
}
// Main function to test the queue operations
int main() {
Q q;
q.front=0;
q.rear=0;
int ch,adhar;
char c[50];
do
{
printf("Enter the choices\n");
printf("1--> Park Cars\t2-->Remove Cars\t3-->Display\n");
scanf("%d",&ch);
switch(ch)
{
case 1: printf("Enter the Car owner adhar number \n");
scanf("%d",&adhar);
```

```
printf("Enter the Car registration number \n");
scanf(" %s",c);
park(&q,adhar,c);
break;
case 2: remove_car(&q);
break;
case 3: display(q);
break;
}
} while(ch!=4);
return 0;
}
```

15	Consider the following structure definition.
	
	struct list1
	{
	int data;
	struct list1 *next;
	};
	
	Write the C functions to implement the following operations on the singly linked list of integers with header node.
	
	Insert: To insert a node into the list in sorted order. The parameters of this function are a pointer to the header node and an integer to be inserted.
	
	RemoveDuplicate: To remove the duplicate element from the list. The parameter of this function is a pointer to the header node.
		3	3	1	3

Ans

Scheme:

C function to implement **Insert** operation – 1.5 M

C function to implement **RemoveDuplicate** operation – 1.5 M

Solution:

```
struct list1
{
   int data;
   struct list1 *next;
};
void insert(struct list1 **head,int d)
{
   struct list1 *newnode,*temp,*prev=NULL;
   newnode=(struct list1 *)malloc(1*sizeof(struct list1 ));
   printf("insede insert ");
    newnode->data=d;
    newnode->next=NULL;
```

```
    if((*head)->next==NULL)
    {

       newnode->next=NULL;
       (*head)->next=newnode;

    }
    else
    {
       for(temp=(*head)->next;temp!=NULL &&(temp->data<d) ;temp=temp->next)
       {
        prev=temp;
       }
       if(prev!=NULL)
       {
          newnode->next=prev->next;
          prev->next=newnode;
       }
       else
       {
          newnode->next=temp;
          (*head)->next=newnode;
       }
        }
}
void  RemoveDuplicate(struct list1 **head)
{
   struct list1 *temp,*temp2;
   temp=(*head)->next;
   while(temp!=NULL)
   {
      if(temp->data==temp->next->data)
      {
         temp2=temp->next;
         temp->next=temp2->next;
         free(temp2);
               }
           temp=temp->next;
   }
   }
```

16	Linked lists find applications in implementation of queues using dynamic memory allocation. To insert an item into the queue, we allocate memory for the new node, store data	3	3	1	3

	in it and add the node at the rear end of the existing queue. To delete an item from the queue, the node at the front of the queue is removed. Assume the typical function calls for ***enqueue()*** and ***dequeue()*** as ***enqueue(&rear, &front, N);*** and ***dequeue(&front, &rear);,*** where N is the element to be added. The structure definition of the node is as given.
	
	// Self- referential structure
	typedef struct node *queue;
	struct node{
	int data;
	Nodeptr next;
	};
	Develop the functions to implement linked implementation of ***enqueue()*** and ***dequeue()*** operations.				

Ans:	**Scheme:**
	Enqueue function – 1.5 M
	Dequeue function – 1.5 M
	
	Solution:
	void enqueue(queue_pointer *front, queue_pointer *rear, element item)
	{ /* add an element to the rear of the queue */
	queue_pointer temp =
	(queue_pointer) malloc(sizeof (queue));
	if (IS_FULL(temp)) {
	fprintf(stderr, " The memory is full\n"); exit(1);
	}
	temp->item = item;
	temp->link = NULL;
	if (*front)
	(*rear) -> link = temp;
	else
	*front = temp;
	*rear = temp;
	}
	40				

```
element dequeue(queue_pointer *front) {
/* delete an element from the queue */
   queue_pointer temp = *front;
   element item;
   if (IS_EMPTY(*front))  {
     fprintf(stderr,  "The queue is empty\n");
     exit(1);
   }
   item = temp->item;
   *front = temp->link;
    free(temp);
    return item;
}
```

17	Develop a C function that accepts a pointer to an array and the size of the array, and reverses the elements of the array in place. Use pointer arithmetic and the dereferencing operator to access and modify array elements.	2	1	1	3

Ans

Scheme
Function with proper arguments and dereferencing – 1M
In place reversing logic – 1M

Solution

```
void reverseArray(int *arr, int size)
{
   int *start = arr;          // Pointer to the beginning of the array
   int *end = arr + size - 1;   // Pointer to the end of the array
   int temp;

   while (start < end)
  {
      // Swap the elements using dereferencing
      temp = *start;
      *start = *end;
      *end = temp;

      // Move the pointers
      start++;
      end--;
   }
}
```

18	Discuss how a stack can help in checking if parentheses in an expression are balanced. Provide an example expression and explain the steps involved in using the stack.	2	2	1	3

Ans	**Scheme:** Steps – 1M Example- 1M **Solution:** Scheme and Solution **1. Initialize an Empty Stack:** • Start with an empty stack. **2. Traverse the Expression:** • Scan the expression from left to right, processing one character at a time. **3. Push Opening Brackets to the Stack:** • Whenever an opening bracket ((, {, or [) is encountered, push it onto the stack. **4. Pop for Matching Closing Brackets:** • Whenever a closing bracket (), }, or]) is encountered, check the stack: o If the stack is empty, it means there is no matching opening bracket, so the expression is unbalanced. o If the stack is not empty, pop the top of the stack. If the popped bracket does not match the current closing bracket, the expression is unbalanced. **5. Final Check:** • After processing the entire expression, check if the stack is empty: o If the stack is empty, the expression is balanced. o If the stack still contains opening brackets, it means some brackets were not closed, and the expression is unbalanced. **Example** Consider the expression: { [( )] } • Step 1: { is an opening bracket, push it onto the stack. o Stack: { • Step 2: [is an opening bracket, push it onto the stack. o Stack: { [• Step 3: (is an opening bracket, push it onto the stack. o Stack: { [(• Step 4:) is a closing bracket, pop from the stack (top is (, matches). o Stack: { [• Step 5:] is a closing bracket, pop from the stack (top is [, matches). o Stack: { • Step 6: } is a closing bracket, pop from the stack (top is {, matches). o Stack: Empty The stack is empty after processing the entire expression, so the parentheses are balanced.				
19	Discuss a priority queue using an unsorted array. Explain how would you perform insertion and deletion operations for the same.	2	2	1	3
Ans	**Scheme:** Insertion – 1M Deletion -1M **Solution:** Scheme and Solution A **priority queue** can be implemented using an unsorted array by storing elements along with their priorities. In this case, the insertion operation is straightforward, but				

deletion (or extraction of the highest/lowest priority element) requires a search through the array.

1. Insertion Operation:

- Insertion in an unsorted array is simple because the new element can be added directly at the end of the array without regard to its priority.

2. Deletion Operation (Extracting Highest/Lowest Priority Element):

- To delete the element with the highest or lowest priority, you must traverse the entire array to find the element with the highest or lowest priority.
- Once found, remove it and possibly shift the elements of the array.

Steps for Each Operation:

Insertion:

- **Algorithm:**
  1. Add the new element at the end of the array.
  2. No need to reorder the array.
- **Example:**
  - Unsorted array: [(3, 'Task A'), (1, 'Task B'), (2, 'Task C')]
  - Insert (4, 'Task D').
  - New array: [(3, 'Task A'), (1, 'Task B'), (2, 'Task C'), (4, 'Task D')]

Deletion (Extract Maximum Priority):

- **Algorithm:**
  1. Traverse the entire array to find the element with the maximum priority.
  2. Remove that element.
  3. Shift the remaining elements if needed (depending on the array structure).
- **Example:**
  - Unsorted array: [(3, 'Task A'), (1, 'Task B'), (2, 'Task C'), (4, 'Task D')]
  - Traverse and find the highest priority element: (4, 'Task D')
  - Remove it: [(3, 'Task A'), (1, 'Task B'), (2, 'Task C')]